Data Visualization Guide: Seaborn & Plotly

1. Introduction

Data visualization is the graphical representation of data to uncover patterns, trends, and insights that are not obvious from raw numbers. In the field of data science, it serves two critical purposes: **exploratory analysis** (understanding data) and **communication** (presenting findings to others). A well-crafted visualization can reveal relationships, outliers, and trends instantly — making it one of the most powerful tools in a data scientist's toolkit.

Python offers a rich ecosystem of visualization libraries, each designed for different use cases. This guide focuses on two of the most widely used and complementary libraries: **Seaborn** and **Plotly**. Together, they cover both **static statistical visualization** and **interactive, presentation-ready graphics** — making them an ideal combination for modern data science workflows.

2. Library Overview

2.1 Seaborn

What it is:
Seaborn is a Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics.

Key Features:

- Built-in themes and color palettes for beautiful defaults
- Works seamlessly with Pandas DataFrames
- Specializes in statistical plots (distributions, correlations, categorical comparisons)
- Less code required compared to Matplotlib for complex plots
- Integrates well with NumPy and Pandas

Typical Use Cases:

- Exploratory Data Analysis (EDA)
- Correlation and distribution analysis
- Comparing categories across groups
- Statistical reporting in notebooks and reports

Best For:
Data scientists who want quick, clean, statistically-focused visuals without writing a lot of code.

2.2 Plotly

What it is:
Plotly is a Python library for creating interactive, web-ready visualizations. It produces charts that support hover, zoom, pan, and click interactions directly in Jupyter notebooks or browsers.

Key Features:

- Fully interactive plots (hover, zoom, pan, click)
- Wide range of chart types including 3D, maps, and animations
- Works in Jupyter, browsers, and web apps (via Dash)
- Exports to HTML for sharing
- Clean, modern default styling
- Supports large datasets efficiently

Typical Use Cases:

- Interactive dashboards and reports
- Presentation-quality charts
- Web applications (with Dash framework)
- Exploratory analysis where interactivity helps
- Geospatial and 3D visualizations

Best For:
Analysts and data scientists who need to present data interactively, build dashboards, or share insights through web-based visualizations.

3. Graph Types

This section documents the most commonly used graph types in Seaborn and Plotly. Each graph includes a description, typical use case, and code example for both libraries.

3.1 Line Plot

Description:
A line plot connects data points with a continuous line, showing trends over a sequence (often time).

Use Case:
Best for time-series data, trends, and comparing changes over intervals.

Code Example (Seaborn):

```
sns.lineplot(x="size", y="total_bill", data=tips)
plt.title("Line Plot")
plt.show()
```

Code Example (Plotly):

```
fig = px.line(tips, x="size", y="total_bill", title="Line Plot")
fig.show()
```

3.2 Scatter Plot

Description:
A scatter plot uses dots to represent values for two numeric variables, showing relationships between them.

Use Case:
Ideal for correlation analysis, outlier detection, and identifying clusters.

Code Example (Seaborn):

```
sns.scatterplot(x="total_bill", y="tip", hue="sex", data=tips)
plt.title("Scatter Plot")
plt.show()
```

Code Example (Plotly):

```
fig = px.scatter(tips, x="total_bill", y="tip", color="sex", title="Scatter Plot")
fig.show()
```

3.3 Bar Chart

Description:
A bar chart represents categorical data with rectangular bars, where the length of each bar corresponds to its value.

Use Case:
Best for comparing quantities across different categories.

Code Example (Seaborn):

```
sns.barplot(x="day", y="total_bill", data=tips)
plt.title("Bar Chart")
plt.show()
```

Code Example (Plotly):

```
avg = tips.groupby("day")["total_bill"].mean().reset_index()
fig = px.bar(avg, x="day", y="total_bill", title="Bar Chart")
fig.show()
```

3.4 Histogram

Description:
A histogram groups continuous data into bins and displays the frequency of values in each bin.

Use Case:
Best for understanding the distribution of a single numeric variable.

Code Example (Seaborn):

```
sns.histplot(tips["total_bill"], bins=20, kde=True)
plt.title("Histogram")
plt.show()
```

Code Example (Plotly):

```
fig = px.histogram(tips, x="total_bill", nbins=20, title="Histogram")
fig.show()
```

3.5 Box Plot

Description:
A box plot summarizes data distribution using quartiles, showing median, interquartile range (IQR), and outliers.

Use Case:
Best for comparing distributions across groups and detecting outliers.

Code Example (Seaborn):

```
sns.boxplot(x="day", y="total_bill", data=tips)
plt.title("Box Plot")
plt.show()
```

Code Example (Plotly):

```
fig = px.box(tips, x="day", y="total_bill", title="Box Plot")
fig.show()
```

3.6 Pie Chart

Description:
A pie chart is a circular graph divided into slices, where each slice represents a proportion of the whole.

Use Case:
Best for showing percentage or proportional distribution across a small number of categories (ideally 5 or fewer).

Code Example (Seaborn):

Note: Seaborn does not have a native pie chart. We use Matplotlib instead.

```
tips["day"].value_counts().plot.pie(autopct="%1.1f%%")
plt.title("Pie Chart")
plt.ylabel("")
plt.show()
```

Code Example (Plotly):

```
fig = px.pie(tips, names="day", title="Pie Chart")
fig.show()
```

3.7 Count Plot

Description:

A count plot displays the number of observations in each category of a categorical variable, using bars.

Use Case:

Best for quickly understanding the frequency distribution of categorical data.

Code Example (Seaborn):

```
sns.countplot(x="day", data=tips)
plt.title("Count Plot")
plt.show()
```

Code Example (Plotly):

```
fig = px.histogram(tips, x="day", title="Count Plot")
fig.show()
```

3.8 Heatmap

Description:

A heatmap uses color intensity to represent values in a matrix, often used to visualize correlation between variables.

Use Case:

Best for correlation analysis, identifying patterns across multiple variables, and spotting relationships.

Code Example (Seaborn):

```
corr = tips[["total_bill", "tip", "size"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Heatmap")
plt.show()
```

Code Example (Plotly):

```
corr = tips[["total_bill", "tip", "size"]].corr()
fig = px.imshow(corr, text_auto=True, color_continuous_scale="RdBu_r",
title="Heatmap")
fig.show()
```

3.9 Violin Plot

Description:
A violin plot combines a box plot with a kernel density plot, showing the full distribution of data across categories.

Use Case:
Best for comparing distributions across groups while also seeing the shape (density) of the data.

Code Example (Seaborn):

```
sns.violinplot(x="day", y="total_bill", data=tips)
plt.title("Violin Plot")
plt.show()
```

Code Example (Plotly):

```
fig = px.violin(tips, x="day", y="total_bill", box=True, title="Violin Plot")
fig.show()
```

3.10 KDE Plot

Description:
A KDE (Kernel Density Estimate) plot shows the probability density of a continuous variable as a smooth curve.

Use Case:
Best for visualizing the distribution shape of continuous data and comparing distributions between groups.

Code Example (Seaborn):

```
sns.kdeplot(data=tips, x="total_bill", hue="sex", fill=True)
plt.title("KDE Plot")
plt.show()
```

Code Example (Plotly):

```
fig = px.histogram(tips, x="total_bill", color="sex", marginal="box", title="KDE-like Plot")
fig.show()
```

---

4. Comparison: Seaborn vs Plotly

Both libraries are powerful, but they serve different purposes. The table below compares them across key criteria to help you choose the right tool for your task.

Criteria Seaborn Plotly

Ease of Use Very easy — simple syntax with smart defaults Easy — slightly more verbose but intuitive **Customization** Moderate — good defaults, limited fine control High — full control over colors, layout, and interactivity **Interactivity** None — static images only Excellent — hover, zoom, pan, click, animations **Performance (Large Data)** Moderate — can slow down with huge datasets Good — handles large data efficiently (WebGL support) **Output Format** Static PNG/SVG (in notebooks) Interactive HTML (browser-ready) **Best For** Statistical EDA, quick insights, reports Dashboards, presentations, web apps **Integration** Seamless with Pandas & NumPy Works with Pandas, Dash, Jupyter **Learning Curve** Low Low to Medium **Community Support** Very large, mature Large, modern, actively growing

When to Use Which?

- **Use Seaborn** when you need fast, clean statistical plots for exploration or static reports. It's perfect for EDA and quick visualization with minimal code.
- **Use Plotly** when you need interactivity, presentation-quality visuals, or dashboards. It shines in web-based applications and sharing insights with non-technical audiences.
- **Use Both** — the ideal workflow: Seaborn for exploration and Plotly for presentation.

---

5. Conclusion

Seaborn and Plotly complement each other perfectly in a modern data science workflow. Seaborn excels at **statistical exploration** with minimal code and beautiful defaults, while Plotly brings **interactivity and presentation-ready visuals** to the table. Together, they cover the full spectrum of visualization needs — from quick EDA to polished dashboards.

By mastering both libraries, data scientists can:

- Explore data efficiently with Seaborn
- Communicate findings interactively with Plotly

- Choose the right tool for each stage of analysis

This guide has documented 10 common graph types with practical code examples for both libraries, providing a solid foundation for building effective visualizations in Python.

---

6. References

- **Seaborn Official Documentation:** https://seaborn.pydata.org/
- **Seaborn Tutorial:** https://seaborn.pydata.org/tutorial/introduction.html
- **Plotly Official Documentation:** https://plotly.com/python/
- **Plotly Express (px) Reference:** https://plotly.com/python/plotly-express/
- **Pandas Documentation:** https://pandas.pydata.org/docs/
- **Matplotlib Documentation:** https://matplotlib.org/stable/users/explain/quick_start.html